

# Dokumentation ToDo-Applikation

## 1. Repository forken und klonen

Zuerst habe ich das vorgegebene Repository auf GitHub geforkt. Dadurch hatte ich eine eigene Kopie des Projekts, an der ich arbeiten konnte.

Mein GitHub-Fork:

<https://github.com/fionabehluli/docker-nodejs-sample.git>

Danach habe ich das Repository lokal in Visual Studio Code geöffnet.

Im Terminal von Visual Studio Code konnte ich mit Git prüfen, ob das Repository korrekt verbunden ist.

## 2. Entwicklungsumgebung

Für die Entwicklung habe ich Visual Studio Code verwendet. Die Befehle habe ich im Git-Terminal ausgeführt.

## 3. README.md

Im README habe ich eine Anleitung erstellt, in der ich meine Schritte aufgeschrieben und erklärt habe

### **ToDo-Applikation Anleitung**

#### **Speicherort Suchen**

Als erstes habe ich einen Ort gesucht, wo die Datei gespeichert werden soll.

#### **GitHub-Fork erstellen**

Zunächst habe ich einen GitHub-Fork erstellt und habe ihn lokal auf meinen Computer geklont. Dafür habe ich auf GitHub das vorgegebene Repository geöffnet und auf Fork geklickt. Anschließend habe ich mein eigenes Repository mit Git geklont und den Projektordner in VS Code geöffnet.

#### **Installation**

- Ich habe alle notwendigen Pakete installiert, indem ich im Terminal "npm install" eingegeben habe.
- Docker desktop hatte ich schon installiert
- Ich habe die APP Node.js installiert. Damit kann die Applikation ausgeführt werden.

#### **ToDo Container**

Ich habe ein Dockerfile erstellt und mit den benötigten Docker Befehlen ausgeführt.

## 4. Dockerisierung der Node.js-Applikation

Für die Dockerisierung habe ich mich an der offiziellen Docker-Anleitung für Node.js orientiert. Dabei habe ich eine Dockerfile, eine Compose-Datei und eine .dockerignore-Datei verwendet. Das Dockerfile beschreibt, wie das Image gebaut wird. Die Compose-Datei beschreibt, wie der Container gestartet wird.

### 4.1 Dockerfile

Auf dem Dockerfile stehen die Schritte, die Docker zum Erstellen des Images benötigt. Dabei werden zuerst die benötigten Dateien kopiert und die npm-Abhängigkeiten installiert. Danach wird der Quellcode in das Image kopiert.

### 4.2 .dockerignore

Mit der .dockerignore-Datei werden Dateien ausgeschlossen, die nicht in das Docker-Image gehören. Dazu gehört zum Beispiel node\_modules.

### 4.3 Docker-Image bauen

Danach habe ich das Docker-Image gebaut. Auf dem Bild kann man sehen, dass das Image erfolgreich erstellt wurde.

### 4.4 Container starten

Beim ersten Versuch war der Port 3000 auf meinem Computer bereits belegt. Deshalb konnte Docker den Port nicht verwenden. Ich habe danach den Host-Port 3001 verwendet und ihn auf den Container-Port 3000 weitergeleitet.

### 4.5 Container prüfen

Mit docker ps kann ich kontrollieren, welche Container laufen. In der Ausgabe kann man den Container mit dem Namen todo-app sehen.

```

=> [5/5] COPY . . 0.6s
=> exporting to image 3.7s
=> => exporting layers 2.1s
=> => exporting manifest sha256:5bd57b432d61bdf0874c331dfb7729549207744a2245da1885 0.0s
=> => exporting config sha256:cd4764b835cbb09e05820844db191d236489e689d9ada8fe73bff 0.0s
=> => exporting attestation manifest sha256:2a99c778fc37a92cff5da11323f2574597ec515 0.0s
=> => exporting manifest list sha256:a3c7d41bfc908eecd452ab7bd22c51bafc924bc6d099f 0.0s
=> => naming to docker.io/library/docker-nodejs-sample:latest 0.0s
=> => unpacking to docker.io/library/docker-nodejs-sample:latest 1.6s

fiona@Fiona MINGW64 /d/040_ZLI/090_Github/docker-nodejs-sample (main)
$ docker run --name todo-app -p 3000:3000 docker-nodejs-sample
docker: Error response from daemon: ports are not available: exposing port TCP 0.0.0.0:3000
-> 127.0.0.1:0: listen tcp 0.0.0.0:3000: bind: Only one usage of each socket address (prot
ocol/network address/port) is normally permitted.

Run 'docker run --help' for more information

What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"

fiona@Fiona MINGW64 /d/040_ZLI/090_Github/docker-nodejs-sample (main)
$ docker.exe ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STA
TUS           PORTS
6d20b67e8d9   docker/welcome-to-docker:latest     "/docker-entrypoint..." 5 hours ago   Up
0 hours      0.0.0.0:8088->80/tcp, [::]:8088->80/tcp  welcome-to-docker

fiona@Fiona MINGW64 /d/040_ZLI/090_Github/docker-nodejs-sample (main)
$ docker.exe ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STA
TUS           PORTS
6d20b67e8d9   docker/welcome-to-docker:latest     "/docker-entrypoint..." 5 hours ago   Up
0 hours      0.0.0.0:8088->80/tcp, [::]:8088->80/tcp  welcome-to-docker

fiona@Fiona MINGW64 /d/040_ZLI/090_Github/docker-nodejs-sample (main)
$ docker run --name todo-app -p 3001:3000 docker-nodejs-sample
docker: Error response from daemon: Conflict. The container name "/todo-app" is already in
use by container "5083b44c45664e0b6a9ba2018fdcfcd3338ac06fcd3fcb7032d72549a79ad280". You ha
ve to remove (or rename) that container to be able to reuse that name.

Run 'docker run --help' for more information

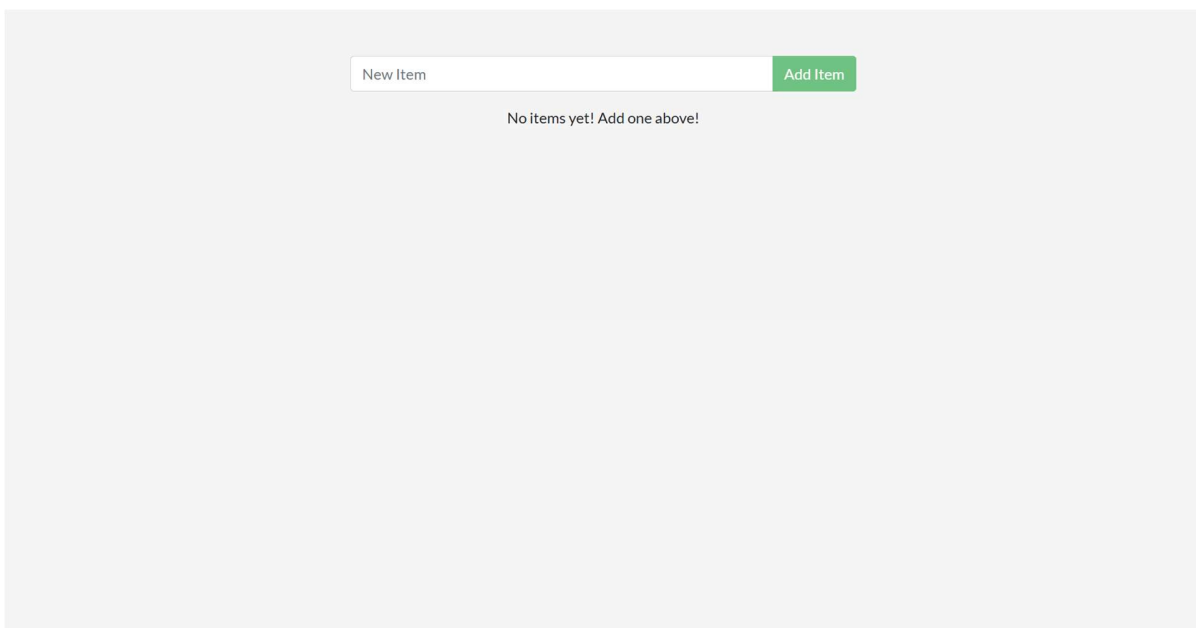
What's next:
  Debug this container error with Gordon → docker ai "help me fix this container error"

fiona@Fiona MINGW64 /d/040_ZLI/090_Github/docker-nodejs-sample (main)
$ docker ps -a
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        S
TATUS        PORTS
083b44c4566   docker-nodejs-sample               "docker-entrypoint.s..." 4 minutes ago   C
reated      todo-app
6d20b67e8d9   docker/welcome-to-docker:latest     "/docker-entrypoint..." 5 hours ago   U

```

## 5. ToDo-Applikation testen

Nachdem der Container gestartet war, habe ich die Anwendung im Browser geöffnet. Die ToDo-Oberfläche wurde angezeigt. Es gibt ein Eingabefeld für neue Aufgaben und einen Knopf, mit dem man Aufgaben hinzufügen kann.



## 6. Git und GitHub

Während der Arbeit habe ich Git verwendet, um die Änderungen am Projekt zu speichern. Mit Git kann ich sehen, welche Dateien geändert wurden und diese Änderungen in einem Commit speichern.

Nach dem Push sind die Änderungen auch in meinem GitHub-Fork vorhanden. Dadurch ist der aktuelle Stand des Projekts online gespeichert.

**Mein GitHub-Fork:** [fionabehluli/docker-nodejs-sample](https://github.com/fionabehluli/docker-nodejs-sample)

## 7. Problem während der Arbeit

Beim Starten des Containers ist zuerst ein Fehler aufgetreten. Der Port 3000 war bereits durch einen anderen Container belegt. Deshalb konnte Docker die Weiterleitung auf Port 3000 nicht erstellen.

Ich habe das Problem gelöst, indem ich den externen Port 3001 verwendet habe. Der Container läuft weiterhin intern auf Port 3000.

Ausserdem musste der Containername „todo-app“ eindeutig sein. Nach dem erfolgreichen Start konnte ich den Container mit `docker ps` überprüfen.

## 8. Fazit

Ich hatte zwar oft Schwierigkeiten damit, die Aufgaben zu verstehen und zum nächsten Schritt zu kommen, doch mit Hilfe von Jan, konnte ich am Ende die ToDo-Applikation erfolgreich in einem Docker-Container starten und im Browser öffnen.

Insgesamt kann ich sagen, es war nicht ganz ohne, aber ich bin einfach froh, dass ich es fertig habe.